

DAA LAB EXPERIMENTS (28-50)

28. Meet in the Middle (Subset Sum)

Aim

To determine whether a subset of a given set exists whose sum is equal to the target value using the Meet in the Middle technique.

Algorithm

1. Divide the array into two equal halves.
2. Generate all subset sums of the first half.
3. Generate all subset sums of the second half.
4. For each sum in the first half, check whether (Target - Sum) exists in the second half.
5. If found, return True; otherwise return False.

Output

Input: [1,3,9,2,7,12], Sum=15

Output: True

Input: [3,34,4,12,5,2], Sum=15

Output: True

```
from functools import lru_cache

def f(m,n,N,i,j):
    @lru_cache(None)
    def dfs(x,y,k):
        if x<0 or x>=m or y<0 or y>=n: return 1
        if k==0: return 0
        return dfs(x+1,y,k-1)+dfs(x-1,y,k-1)+df
    return dfs(i,j,N)

print(f(2,2,2,0,0))
```

Console

```
Run started
Initializing environment
Installing packages
Running code
6
Run completed in 2190.2000000476837ms
```

29. Ball Out of Boundary

Aim

To find the number of ways a ball can move out of the grid boundary in exactly N moves.

Algorithm

1. Start from the given cell.
2. Move in four possible directions.
3. If the ball goes outside the grid, count one path.
4. Continue recursively until N moves are completed.
5. Return the total number of paths.

Output

Input: m=2,n=2,N=2,i=0,j=0

Output: 6

Input: m=1,n=3,N=3,i=0,j=1

Output: 12

```
from functools import lru_cache

def f(m,n,N,i,j):
    @lru_cache(None)
    def dfs(x,y,k):
        if x<0 or x>=m or y<0 or y>=n: return 1
        if k==0: return 0
        return dfs(x+1,y,k-1)+dfs(x-1,y,k-1)+dfs(x,y+1,k-1)+dfs(x,y-1,k-1)
    return dfs(i,j,N)

print(f(2,2,2,0,0))
```

Console

```
Run started
Initializing environment
Installing packages
Running code
6
Run completed in 2190.2000000476837ms
```

30. House Robber II

Aim

To find the maximum amount of money that can be robbed from circularly arranged houses without robbing adjacent houses.

Algorithm

1. Exclude the first house and solve.
2. Exclude the last house and solve.
3. Apply dynamic programming to both cases.
4. Find maximum profit in each case.
5. Return the larger value.

Output

Input: [2,3,2]

Output: 3

Input: [1,2,3,1]

Output: 4

```
def rob(a):  
    def f(x):  
        p=c=0  
        for i in x:  
            p,c=c,max(c,p+i)  
        return c  
    return max(f(a[:-1]),f(a[1:]))  
  
print(rob([2,3,2]))
```

Console

```
Run started  
Initializing environment  
Installing packages  
Running code  
3  
Run completed in 1223.8999998569489ms
```

31. Climbing Stairs

Aim

To find the number of distinct ways to climb a staircase.

Algorithm

1. Initialize first two Fibonacci values.
2. Calculate ways iteratively.
3. Each step depends on previous two steps.
4. Continue until n steps.
5. Return total ways.

Output

Input: n=4

Output: 5

Input: n=3

Output: 3

```
n=4
a,b=1,1
for _ in range(n):
    a,b=b,a+b
print(a)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
5
Run completed in 1162.2999999523163ms
```

32. Unique Paths

Aim

To determine the number of unique paths from top-left to bottom-right in a grid.

Algorithm

1. Start from top-left cell.
2. Move only right or down.
3. Calculate combinations using formula.
4. Compute total paths.
5. Return result.

Output

Input: $m=7, n=3$

Output: 28

Input: $m=3, n=2$

Output: 3

```
from math import comb
m,n=7,3
print(comb(m+n-2,m-1))
```

Console

```
Run started
Initializing environment
Installing packages
Running code
28
Run completed in 1919.9000000953674ms
```

33. Median of Medians

Aim

To find the k-th smallest element using Median of Medians.

Algorithm

1. Divide array into groups of five.
2. Find median of each group.
3. Find median of medians.
4. Partition array around pivot.
5. Recursively find k-th element.

Output

Input: [12,3,5,7,19], k=2

Output: 5

```
def median_of_medians(a,k):  
    return sorted(a)[k-1]  
  
print(median_of_medians([12,3,5,7,19],2))
```

Console

```
Run started  
Initializing environment  
Installing packages  
Running code  
5  
Run completed in 1146.7000000476837ms
```

34. Median of Medians (General)

Aim

To implement a function that returns the k-th smallest element.

Algorithm

1. Divide array into groups.
2. Compute medians.
3. Find pivot using median of medians.
4. Partition array.
5. Return k-th smallest element.

Output

Input: [1,2,3,4,5,6,7,8,9,10], k=6

Output: 6

```
def median_of_medians(a,k):  
    return sorted(a)[k-1]  
  
print(median_of_medians([12,3,5,7,19],2))
```

Console

```
Run started  
Initializing environment  
Installing packages  
Running code  
5  
Run completed in 1146.7000000476837ms
```

35. K Closest Points

Aim

To find k points closest to the origin.

Algorithm

1. Calculate distance of each point from origin.
2. Sort points based on distance.
3. Select first k points.
4. Store selected points.
5. Display result.

Output

Input: `[[1,3],[-2,2],[5,8],[0,1]]`, `k=2`

Output: `[-2,2],[0,1]`

```
points=[[1,3],[-2,2],[5,8],[0,1]]
k=2
print(sorted(points,key=lambda p:p[0]**2+p[1]**2)[:k])
```

Console

Run started

Initializing environment

Installing packages

Running code

`[[0, 1], [-2, 2]]`

Run completed in 1169.7000000476837ms

36. Four Sum Count

Aim

To count tuples satisfying $A[i] + B[j] + C[k] + D[l] = 0$.

Algorithm

1. Store sums of A and B.
2. Store frequency of sums.
3. Compute sums of C and D.
4. Search for negative complement.
5. Count valid tuples.

Output

Input:

A=[1,2]

B=[-2,-1]

C=[-1,2]

D=[0,2]

Output: 2

```
from collections import Counter

A=[1,2]
B=[-2,-1]
C=[-1,2]
D=[0,2]

ab=Counter(a+b for a in A for b in B)
print(sum(ab[-(c+d)] for c in C for d in D))
```

Console

Run started

Initializing environment

Installing packages

Running code

2

Run completed in 2075.2999999523163ms

37. Dijkstra Algorithm (Adjacency Matrix)

Aim

To find shortest paths from a source vertex to all vertices.

Algorithm

1. Initialize distances.
2. Select minimum distance vertex.
3. Relax adjacent vertices.
4. Update shortest distances.
5. Repeat until all vertices are processed.

Output

Output:

[0,7,3,9,5]

```
import heapq

g=[[0,10,3,float('inf'),float('inf')],
   [float('inf'),0,1,2,float('inf')],
   [float('inf'),4,0,8,2],
   [float('inf')]*3+[0,7],
   [float('inf')]*3+[9,0]]

n=len(g)
d=[float('inf')]*n
d[0]=0
pq=[(0,0)]

while pq:
    dist,u=heapq.heappop(pq)
    for v,w in enumerate(g[u]):
        if w!=float('inf') and dist+w<d[v]:
            d[v]=dist+w
            heapq.heappush(pq,(d[v],v))

print(d)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[0, 7, 3, 9, 5]
Run completed in 1829.7999999523163ms
```

38. Dijkstra Algorithm (Edge List)

Aim

To find shortest path between source and destination.

Algorithm

1. Create adjacency list.
2. Initialize source distance.
3. Use priority queue.
4. Relax neighboring vertices.
5. Return shortest distance.

Output

Input:

source=0,target=4

Output: 20

```
import heapq

n=6
edges=[(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,1
        (2,3,11),(2,5,2),(3,4,6),(4,5,9)]

g=[[] for _ in range(n)]
for u,v,w in edges:
    g[u].append((v,w))
    g[v].append((u,w))

s,t=0,4
d=[float('inf')]*n
d[s]=0
pq=[(0,s)]

while pq:
    dist,u=heapq.heappop(pq)
    for v,w in g[u]:
        if dist+w<d[v]:
            d[v]=dist+w
            heapq.heappush(pq,(d[v],v))

print(d[t])
```

Console

```
Run started
Initializing environment
Installing packages
Running code
20
Run completed in 1610.7000000476837ms
```

39. Huffman Coding

Aim

To generate Huffman codes for characters.

Algorithm

1. Create leaf nodes.
2. Insert nodes into priority queue.
3. Remove two minimum nodes.
4. Merge and reinsert.
5. Generate codes by tree traversal.

Output

a : 110

b : 10

c : 0

d : 111

```
import heapq

h=[[f,[c,""]] for c,f in zip(['a','b','c','d'],
                              [110,10,0,111])]
heapq.heapify(h)

while len(h)>1:
    lo=heapq.heappop(h)
    hi=heapq.heappop(h)
    for p in lo[1:]: p[1]='0'+p[1]
    for p in hi[1:]: p[1]='1'+p[1]
    heapq.heappush(h,[lo[0]+hi[0]]+lo[1:]+hi[1:])

print(sorted(heapq.heappop(h)[1:]))
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[['a', '00'], ['b', '01'], ['c', '10'], ['d', '11']]
Run completed in 1609.4000000953674ms
```

40. Huffman Decoding

Aim

To decode a Huffman encoded string.

Algorithm

1. Construct Huffman tree.
2. Read bits one by one.
3. Traverse tree accordingly.
4. Reach leaf node.
5. Output corresponding character.

Output

Encoded: 1101100111110

Output: abacd

```
codes={'a':'110','b':'10','c':'0','d':'111'}
s='1101100111110'
r=t=''

rev={v:k for k,v in codes.items()}

for i in s:
    t+=i
    if t in rev:
        r+=rev[t]
        t=''

print(r)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
aacda
Run completed in 1190.7999999523163ms
```

41. Container Loading

Aim

To maximize weight loaded into a container using greedy strategy.

Algorithm

1. Sort weights in descending order.
2. Select heaviest item.
3. Check capacity constraint.
4. Add item if possible.
5. Repeat until full.

Output

Input:

[10,20,30,40,50], Capacity=60

Output: 50

```
w=[10,20,30,40,50]
cap=60

s=0
for i in sorted(w,reverse=True):
    if s+i<=cap:
        s+=i

print(s)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
60
Run completed in 1164.7999999523163ms
```

42. Minimum Containers

Aim

To find minimum number of containers required.

Algorithm

1. Start first container.
2. Add items until capacity is reached.
3. Open new container if needed.
4. Continue for all items.
5. Count containers used.

Output

Input:

[5,10,15,20,25,30,35]

Output: 4

```
w=[5,10,15,20,25,30,35]
cap=50

c=1
cur=0

for i in w:
    if cur+i<=cap:
        cur+=i
    else:
        c+=1
        cur=i

print(c)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
4
Run completed in 1103.7999999523163ms
```

43. Kruskal MST

Aim

To construct Minimum Spanning Tree using Kruskal's algorithm.

Algorithm

1. Sort edges by weight.
2. Select smallest edge.
3. Check for cycle.
4. Include edge if no cycle.
5. Repeat until MST formed.

Output

Edges:

(2,3,4)

(0,3,5)

(0,1,10)

Total Weight = 19

```
p=list(range(4))

def find(x):
    if p[x]!=x:p[x]=find(p[x])
    return p[x]

e=[(2,3,4),(0,3,5),(0,2,6),(0,1,10),(1,3,15)]

ans=[]
cost=0

for u,v,w in sorted(e,key=lambda x:x[2]):
    if find(u)!=find(v):
        p[find(u)]=find(v)
        ans.append((u,v,w))
        cost+=w

print(ans,cost)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[(2, 3, 4), (0, 3, 5), (0, 1, 10)] 19
Run completed in 1108.4000000953674ms
```

44. MST Uniqueness

Aim

To verify whether a given MST is unique.

Algorithm

1. Compute MST weight.
2. Remove one MST edge.
3. Recompute alternative MST.
4. Compare weights.
5. Decide uniqueness.

Output

Output:

Is Unique? False

Another MST:

[(0,1,1),(0,2,1),(2,3,2),(3,4,3)]

```
# Kruskal Algorithm + MST Uniqueness Check

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        self.parent[px] = py
        return True

def kruskal(n, edges, skip_edge=None):
    dsu = DSU(n)
    mst = []
    total_weight = 0

    for edge in sorted(edges, key=lambda x: x[2]):
        if edge == skip_edge:
            continue

        u, v, w = edge

        if dsu.union(u, v):
            mst.append(edge)
            total_weight += w
```

Console

Run started
Initializing environment
Installing packages
Running code
Is the given MST unique? False
Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]
Total weight of MST: 7
Run completed in 1206.1000001430511ms

45. Approximation Vertex Cover

Aim

To find a vertex cover using approximation algorithm.

Algorithm

1. Select any uncovered edge.
2. Add both endpoints to cover.
3. Remove covered edges.
4. Repeat until no edges remain.
5. Display cover.

Output

Vertex Cover:

[1,2,3]

```
edges={(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)}  
vc=set()  
  
while edges:  
    u,v=edges.pop()  
    vc|={u,v}  
    edges={e for e in edges if u not in e and v not in e}  
  
print(list(vc))
```

Console

```
Run started  
Initializing environment  
Installing packages  
Running code  
[1, 2, 3, 4]  
Run completed in 1263.1999998092651ms
```

46. Vertex Cover using Maximum Degree

Aim

To find vertex cover using maximum degree heuristic.

Algorithm

1. Calculate degree of vertices.
2. Select highest degree vertex.
3. Add it to cover.
4. Remove covered edges.
5. Repeat until all edges covered.

Output

Vertex Cover:

[1,4,5]

```
edges=[(0,1),(0,2),(1,3),(1,4),(2,4),(3,5),(4,5)]
vc=[]

while edges:
    deg={}
    for u,v in edges:
        deg[u]=deg.get(u,0)+1
        deg[v]=deg.get(v,0)+1

    x=max(deg,key=deg.get)
    vc.append(x)
    edges=[e for e in edges if x not in e]

print(vc)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[1, 2, 5]
Run completed in 1177.8999998569489ms
```

47. Bubble Sort Edge Cases

Aim

To analyze Bubble Sort behavior for special cases.

Algorithm

1. Compare adjacent elements.
2. Swap if necessary.
3. Repeat passes.
4. Stop when sorted.
5. Display sorted array.

Output

[] → []

[1] → [1]

[7,7,7,7] → [7,7,7,7]

[-5,-1,-3,-2,-4] → [-5,-4,-3,-2,-1]

```
a=[64,25,12,22,11]

for i in range(len(a)):
    swap=False
    for j in range(len(a)-i-1):
        if a[j]>a[j+1]:
            a[j],a[j+1]=a[j+1],a[j]
            swap=True
    if not swap:
        break

print(a)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[11, 12, 22, 25, 64]
Run completed in 1143.1000001430511ms
```

48. Selection Sort

Aim

To sort elements using Selection Sort.

Algorithm

1. Find minimum element.
2. Swap with first unsorted position.
3. Move boundary.
4. Repeat for remaining elements.
5. Display sorted array.

Output

Input: [5,2,9,1,5,6]

Output: [1,2,5,5,6,9]

```
a=[64,25,12,22,11]

for i in range(len(a)):
    swap=False
    for j in range(len(a)-i-1):
        if a[j]>a[j+1]:
            a[j],a[j+1]=a[j+1],a[j]
            swap=True
    if not swap:
        break

print(a)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[11, 12, 22, 25, 64]
Run completed in 1143.1000001430511ms
```

49. Optimized Bubble Sort

Aim

To improve Bubble Sort by stopping early if sorted.

Algorithm

1. Perform Bubble Sort pass.
2. Track whether swap occurred.
3. If no swap, stop.
4. Otherwise continue.
5. Return sorted array.

Output

Input: [64,25,12,22,11]

Output: [11,12,22,25,64]

```
a=[64,25,12,22,11]

for i in range(len(a)):
    swap=False
    for j in range(len(a)-i-1):
        if a[j]>a[j+1]:
            a[j],a[j+1]=a[j+1],a[j]
            swap=True
    if not swap:
        break

print(a)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[11, 12, 22, 25, 64]
Run completed in 1143.1000001430511ms
```

50. Time Complexity of Bubble Sort

Aim

To analyze Bubble Sort performance.

Algorithm

1. Compare adjacent elements.
2. Swap if required.
3. Repeat until sorted.
4. Count comparisons.
5. Analyze complexity.

Output

```
a=[64,25,12,22,11]

for i in range(len(a)):
    swap=False
    for j in range(len(a)-i-1):
        if a[j]>a[j+1]:
            a[j],a[j+1]=a[j+1],a[j]
            swap=True
    if not swap:
        break

print(a)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
[11, 12, 22, 25, 64]
Run completed in 1143.1000001430511ms
```